

Nutika Elektrivõrgu Juhtimiskeskus: Täislahendus

Käesolev tehniline ülesanne (TÜ) kirjeldab täisfunktsionaalse fullstack veebirakenduse arendamist, mille eesmärk on intelligentne nutiseadmete elektritarbimise haldamine vastavalt Nord Pooli reaalajas börsihindadele. Dokument on koostatud üliõpilase IT-projekti alusdokumendina ning katab kõiki olulisi aspekte projekti visioonist kuni infrastruktuuri seireni.

Rakendus integreerib börsihindade automaatse jälgimise, seadmete kaugjuhtimise ning kasutajasõbraliku juhtimispaneeli ühtseks terviklahenduseks. Projekt on mõeldud nii igapäevaseks koduseks kasutamiseks kui ka väiksemate ärikeskkondade energiahalduseks, pakkudes olulist kokkuhoidu elektriarvetes ning suurendades teadlikkust energiatarbimisest.

6 Põhijaotist

Visioon, nõuded, moodulid, lisafunktsioonid, testimine, infrastruktuur

Fullstack Lahendus

Täielik veebirakendus front-end, back-end ja andmebaasiga

Self-hosted CI/CD

Automaatne deploy Coolify platvormil GitHub App integratsiooniga

Projekti Visioon

JAOTIS 1

Projekti keskne eesmärk on luua intelligentne juhtimissüsteem, mis seob reaajas Nord Pooli elektribörsi hinnaandmed koduautomaatika nutiseadmetega. Süsteem jälgib pidevalt hetkehindu ning reageerib automaatselt eelnevalt kasutaja poolt seadistatud hinnapiiridele — kui börsihind ületab määratud lävendi, lülitatakse mittekritilised tarbijad automaatselt välja, ning hinna langedes taastuvad seadmed tööolekusse. Selline lähenemisviis võimaldab elektrikulu optimeerida passiivselt ilma kasutaja pideva sekkumiseta.

Rakendus on kavandatud fullstack veebirakendusena, mis tähendab, et kõik komponendid — kasutajaliides, äriloogika serveripool ning andmete püsisalvestus — on integreeritud ühtse ja koheselt deployitava lahendusena. Kasutajaliides peab olema intuitiivne ning reageeriv, toimides sujuvalt nii lauaarvutis kui ka mobiilseadmetes. Serveripoolne loogika vastutab börsihindade perioodilise tõmbamise, seadmetele käskude saatmise ning ajalooliste andmete salvestamise eest.

Lahenduse arhitektuuriline printsiip tugineb selgele kihistusele: esitluskiht (UI), äriloogikakiht (API/backend) ning andmekiht (andmebaas ja välised integratsioonid). Iga kiht peab olema arendatud autonoomselt testitavaks ja sõltumatuks, mis lihtsustab hooldust, testimist ning tulevasi täiendusi. Kogu kood tuleb hoida GitHubis, kus GitHub App tagab automaatse CI/CD pipeline'i ning sujuva deploymentu Coolify platvormil.

Peamine väärtuspakkumine

Automaatne energiasääst Nord Pooli hinnaandmete põhjal — kasutaja seadistab reegli üks kord ning süsteem optimeerib tarbimist iseseisvalt ööpäevaringselt.

Sihtrühm

- Koduomanikud nutiseadmetega
- Väikeettevõtted börsihinnaga elektrilepinguga
- Energiateadlikud kasutajad, kes soovivad kulusid vähendada
- IT-tudengid, kes soovivad reaalse projekti kogemust

Tehnilised Nõuded ja Infrastruktuur

JAOTIS 2

Kogu rakenduse tehniline alus peab vastama kaasaegsetele standarditele. Kõik kasutatud teegid, raamistikud ja sõltuvused (dependencies) peavad olema uusimad stabiilsed versioonid — aegunud pakettide kasutamine ei ole lubatud ning see peab olema kontrollitav automaatse dependency audit tööriistaga (nt `npm audit` või `pip-audit`). Igasugune teadlik vananenud versiooni kasutamine nõuab kirjalikku põhjendust.

Rakenduse majutusplatvormina kasutatakse enesemajutatud (self-hosted) Coolify instantsi. Coolify on avatud lähtekoodiga PaaS-lahendus, mis võimaldab hallata konteineripõhiseid rakendusi ilma keerulise infrastruktuuri seadistusest. Arendaja peab tagama, et rakendus on pakendatud Dockeri konteinerisse koos korrektse `docker-compose.yml` või samaväärse konfiguratsioonifailiga, mis võimaldab Coolifyl rakendust automaatselt üles ehitada ja käivitada.

CI/CD pipeline tuleb seadistada läbi GitHub App integratsiooni. Iga push main-harusse käivitab automaatselt testide jooksumise, Docker image'i ehitamise ning uue versiooni deploymentu Coolify keskkonda. Feature-harudes peab pipeline käivitama vähemalt testid ja linting-kontrolli, kuid automaatset deploymentu ei nõuta. Kõik keskkonna muutujad (API võtmed, andmebaasi ühendusandmed) peavad olema hallatavad Coolify seadetes, mitte kõvakodeeritud repositooriumis.

Sõltuvuste haldus

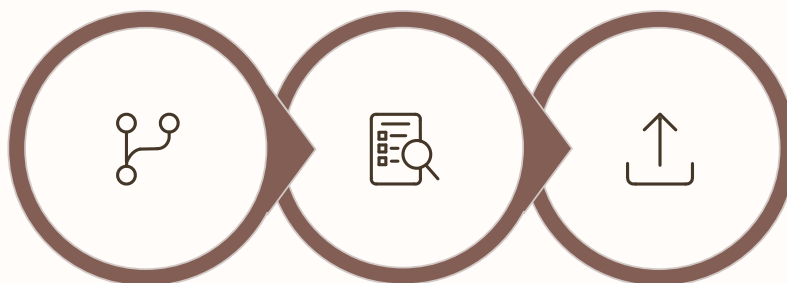
Kõik pakettid uusimas stabiilses versioonis. Automaatne `npm audit` / `pip-audit` CI pipeline'is. Lock-failid (`package-lock.json`, `poetry.lock`) peavad olema repositooriumis.

Coolify Majutus

Self-hosted Coolify instants. Dockeri konteineripõhine deploy. Konfiguratsioon `docker-compose.yml` failis. Keskkonna muutujad Coolify secrets-halduris.

GitHub App CI/CD

Automaatne pipeline igal push-sündmusel. Testid → Build → Deploy järjestus. Feature-harudes: testid ja linting. Main-harul: täielik deploy.



Kood GitHubi

Automaatsed
testid

Deploy
Coolify'sse

Ülaltoodud pipeline tagab, et iga koodimuutus läbib automaatse kvaliteedikontrolli enne tootmiskeskonda jõudmist, minimeerides inimlikku eksimust ja tagades usaldusväärse tarneprotsessi.

Funktsionaalsed Moodulid

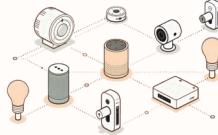
JAOTIS 3

Rakendus koosneb kolmest põhimoodulist, millest igaüks täidab selgelt piiritletud rolli kogu süsteemi toimimises. Moodulite vahel on hästi defineeritud liidesed ning iga moodul peab olema arendatav ja testitav sõltumatult. Alljärgnevalt on kirjeldatud iga mooduli funktsionaalsed nõuded detailsetl.



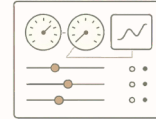
Kasutajahaldus (User Master)

Süsteem peab toetama mitme kasutaja paralleelset kasutust. Peakasutaja (Master) rollil on täielikud haldusõigused: teiste kasutajate lisamine, muutmine, deaktiveerimine ja eemaldamine. Tavakasutatajal on juurdepääs vaid oma seadmetele ja andmetele. Autentimine peab olema turvaline (JWT või seansitoken), paroolid bcrypt/argon2 räsitud. Kasutajarollide kontroll peab toimuma nii front-endis kui ka backend API-s.



Seadmete Haldus

Kasutaja saab lisada nutiseadmeid, määrates igale unikaalse nime, kirjelduse ja ühendusparameetrid (IP-aadress, API endpoint või MQTT teema). Seadmeid saab muuta ja eemaldada. Iga seade peab toetama sisse/välja lülitamise signaali saatmist. Süsteem peab logima kõik seadmele saadetud käsud koos ajatempliga ning olema võimeline pärima seadme hetkeolekut. Seadme lisamisel peab toimuma ühenduse testimine.



Juhtimiskeskus (Control Center)

Visuaalne reaalajas paneel on rakenduse südamik. Kasutaja näeb hetkel kehtivat börsihinda, kõigi seadmete olekut ning saab teostada käsitsi ülekirjutusi (manual override). Igale seadmele saab seadistada individuaalse hinnapiiri (threshold eurodes/MWh). Börsihind uueneb automaatselt ilma lehte värskendamata (WebSocket või polling). Override-režiimis seade ignoreerib automaatikat kuni kasutaja selle manuaalselt lõpetab.

- i** Kõik kolm moodulit peavad olema omavahel integreeritud ühtseks andmevooks: Juhtimiskeskus kasutab Seadmete halduse andmeid ning Kasutajahaldus kontrollib, millistele seadmetele kasutajal juurdepääs on.

Lisafunktsioonid Kliendi jaoks

JAOTIS 4

Lisaks põhimoodulitele peab rakendus pakkuma kliendile lisaväärtust andvaid funktsioone, mis muudavad süsteemi kasutamise mugavamaks, informatiivsemaks ning annavad selge ülevaate saavutatud kokkuhoiust. Need funktsioonid ei ole pelgalt "nice-to-have" — need on lahutamatu osa tootepakkumisest ning peavad olema täielikult implementeeritud ja kaetud testidega.



Säästuaruanne

Arvutab ja visualiseerib, kui palju eurosid on automaatjuhtimise süsteem säästnud võrreldes fikseeritud elektripaketiga. Näitab statistikat päeva, nädala ja kuu lõikes. Säästuarvestuse algoritm peab olema selgelt dokumenteeritud ja ühiktestidega kaetud.



Tulevikuproгноos

Visualiseerib järgmise 24 tunni Nord Pooli börsihindade prognoosi koos süsteemi planeeritud automaatsete tegevustega — millistel tundidel milliseid seadmeid sisse või välja lülitatakse. Prognoos tõmmatakse Elingi avalikust API-st.



Nutikad Teavitused

Saadab reaajas teavitusi Telegrami boti või Discordi webhook'i kaudu: börsihind ületab kriitilise piiri, seade lülitub sisse/välja automaatselt, või seadme ühendus katkeb. Kasutaja saab ise valida teavituskanali ja kriitilisuse lävendid.



Puhkuserežiim

Ühe nupuvajutusega aktiveeritav režiim, mis deaktiveerib kõik kasutaja poolt mitte-kriitiliseks märgitud seadmed. Kriitilised seadmed (nt külmkapp) jäävad tööle. Regime'ist väljumine taastab kõik eelmised seadistused automaatselt.

Teavituskanalite nõuded

Rakendus peab toetama **vähemalt ühte** järgnevatest teavituskanalitest täielikult toimiva implementatsiooniga. Mõlema toetamine annab lisapunkte:

- Telegram Bot API (sõnumid kasutaja chatile)
- Discord Webhook (sõnumid serverikanali)

Säästuaruande arvestusmetoodika

Võrdlusaluseks kasutatakse kasutaja poolt sisestatavat fikseeritud paketi hinda (€/kWh). Süsteem arvutab reaalselt makstud kulu (tegeliku börsihinna alusel seadme tööaja põhjal) ning kuvab vahe eurosummana ja protsentides. Arvestus peab olema läbipaistev — kasutaja saab näha iga perioodi detailset arvutust.

Testimine ja Töökindlus

JAOTIS 5

Usaldusväärne süsteem ei ole ainult see, mis töötab ideaalsetel tingimustel — see on süsteem, mis käitub korrektselt ka vigaste sisendite, katkiste ühenduste ja ootamatute andmete korral. Käesolev jaotis kirjeldab testimise ja töökindluse nõudeid, millele rakendus peab vastama enne lõplikku hindamist. Testimata kood ei ole aktsepteeritav tootmiskeskkonda saadetuna.

1 Sisendite Valideerimine ja Veahaldus

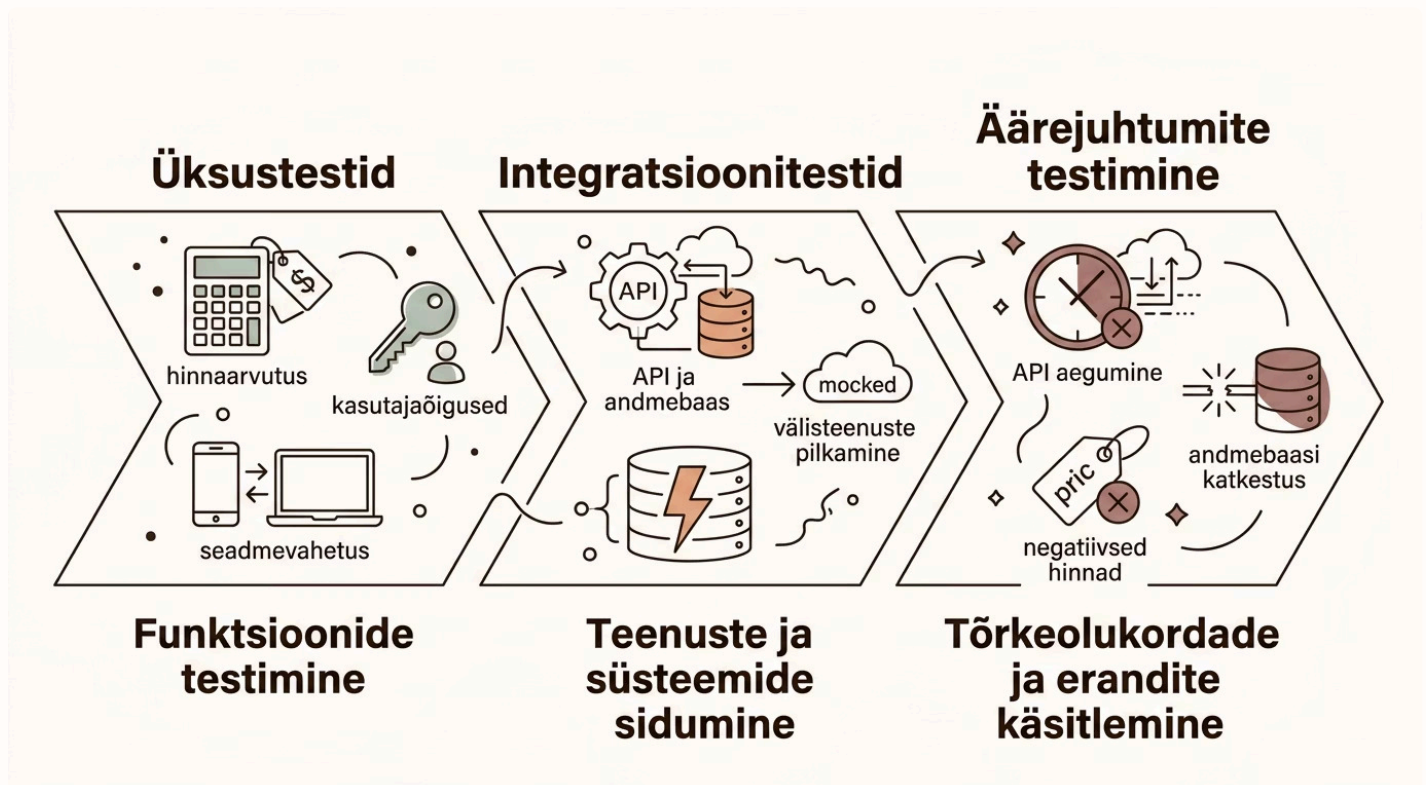
Iga kasutajalt tulev sisend (vormid, URL-parameetrid, JSON-body) peab läbima range serveripoolse valideerimise sõltumata front-end valideerimisest. API päringud välisele Eleringi teenusele peavad käsitlema timeout'e, vigaseid vastuseid ja võrguprobleeme graceful degradation põhimõttel — kasutajale kuvatakse informatiivne veateade, mitte stack trace ega "500 Internal Server Error". Rakendus ei tohi mingis olukorras täielikult kokku kukkuda (crash) vigaste sisendandmete tõttu.

2 Automaattestid

Järgmised äriloogika komponendid peavad olema kaetud nii ühik- (unit) kui ka integratsioonitestidega: hinnapiiri arvutusalgoritm ja seadme lülitusotsus, säästuaruande kalkulaator, kasutajaõiguste kontroll (autoriseerimine), ning seadmele käsu saatmise loogika. Testimise katvus (*code coverage*) peab olema vähemalt 80% kriitiliste moodulite puhul. Testid käivituvad automaatselt CI pipeline'is.

3 Kõrvalekallete (Edge-Case) Haldamine

Süsteemil peab olema defineeritud ja implementeeritud käitumine järgmisteks olukordadeks: Eleringi API ei vasta → kasutatakse viimast teadaolevat hinda koos hoiatussõnumiga; andmebaasi ühendus katkeb → rakendus logib vea, kuvab kasutajale staatuse ning ei saada automaatseid käske; elektri börsihind langeb alla nulli → süsteem ei lähe silmusesse ega lülita kõiki seadmeid käitumisloogi poolest oodatult ümber, vaid rakendab eelnevalt defineeritud reeglit.



⚠ Testimata kriitilise äriloogikaga projekt ei läbi lõplikku hindamist. Ühiktestid peavad olema jooksutavad käsuga ilma väliste sõltuvusteta (mock'itud API-d ja andmebaas).

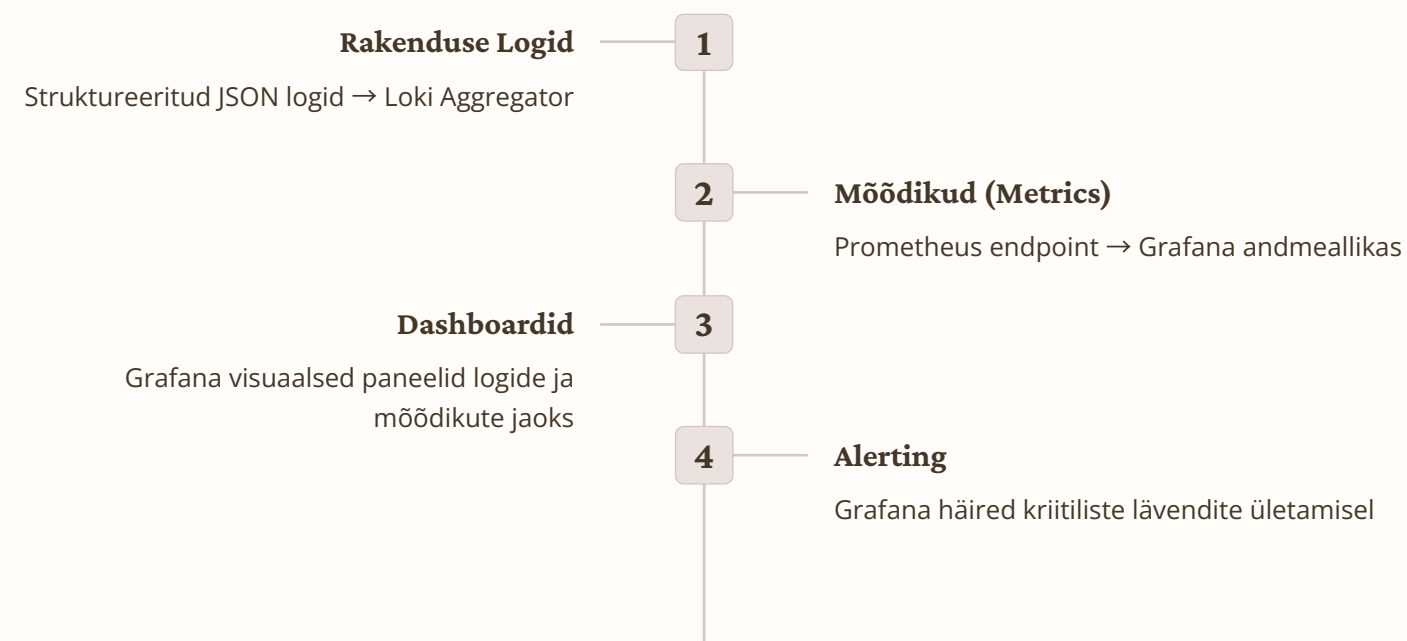
Infrastruktuur ja Seire

JAOTIS 6

Tootmiskeskonda deployitud rakendus peab olema monitooritav ja logitav. Ilma korraliku seiresüsteemita on võimatu tuvastada jõudlusprobleeme, tuvastada anomaaliaid või diagnoosida vigade põhjuseid. Käesolev projekt nõuab Grafana ja Loki tööriistade integreerimist Coolify keskkonnaga, mis loob tervikliku seire- ja logimisplatvormi.

Grafana on avatud lähtekoodiga andmevisualiseerimise ja monitooringu platvorm. Rakendus peab eksportima mõõdikuid (metrics) Prometheus-ühilduvas formaadis, mida Grafana saab tarbida. Arendaja peab looma vähemalt ühe Grafana dashboardi, mis kuvab järgmisi mõõdikuid reaajas: API päringute arv ja vastamise kiirus (latency), seadmele saadetud käskude arv ajavahemikus, aktiivsete kasutajasessioonide arv ning süsteemi mälu- ja protsessorikoormus.

Loki on logide agregeerimissüsteem, mis on loodud Grafanaga töötamiseks. Kõik rakenduse logid (info, warning, error) peavad olema struktureeritud JSON-formaadis ning saadetud Loki instansi. See võimaldab logide filtreerimist ja pärimist Grafana liidestest. Eriti oluline on see, et kõik edge-case'i olukorrad (Eleringi API timeout, DB ühenduse katkestus, negatiivne elektri hind) peavad genereerima selgelt identifitseeritava error-taseme logi koos kontekstuaalse informatsiooniga.



Kohustuslikud Grafana Dashboardid

- API jõudlus: päringute latency P50/P95/P99
- Seadmete aktiivsus: käskude ajalugu graafikul
- Vigade määr: error-logide arv ajas
- Süsteemi ressursid: CPU, RAM, võrguliiklus
- Börsihinna ajalugu koos seadmete olekuga

Logi Tasemed ja Nõuded

- **INFO:** Seadme lülitamine, kasutaja sisselogimine, API päring
- **WARNING:** Eleringi API aeglane vastus, kõrge elektri hind
- **ERROR:** API timeout, DB ühenduse kadu, autentimise viga
- **CRITICAL:** Süsteemi käivitamise tõrge, andmete korrumpeerumine

✔ Grafana ja Loki seadistamine näitab DevOps pädevust ning on projekti hindamisel lisaväärtuseks. Hästi seadistatud seiresüsteem on märk professionaalsest lähenemisest tootekvaliteedile.